

IPSEC-VPN

Technical Requirements Document (TRD)

August 31, 2026

Technical Requirements Document (TRD)

Project: IPSEC-VPN — AI-Powered IPsec VPN Protocol Analyzer

1. Architecture Overview
2. Technology Stack
3. Service Breakdown
4. Data Model (PostgreSQL — core tables)
5. Redis Usage
6. API Design (representative REST endpoints, Go backend)
7. AI/ML Pipeline Details
8. Security Assessment Rules Engine
9. Deployment Architecture
10. Security Considerations (of the platform itself)
11. Testing Strategy
12. Milestone Mapping to PRD Phases

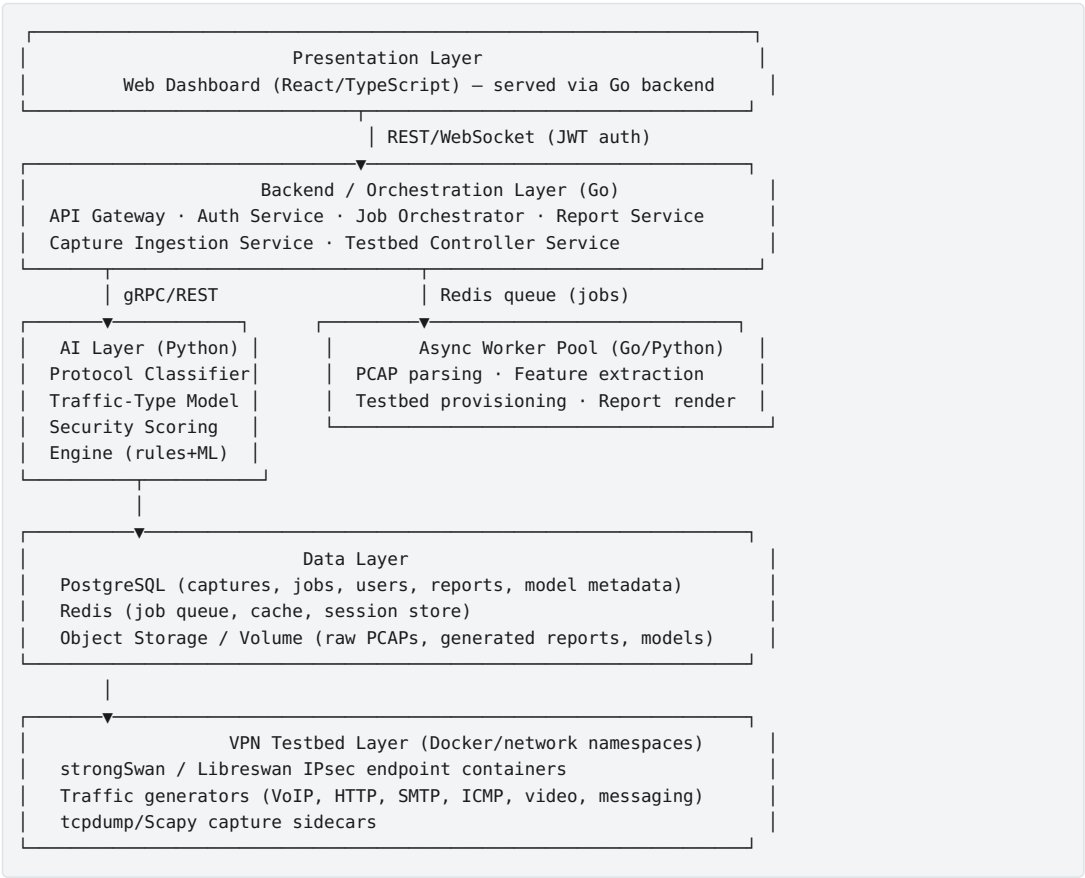
Technical Requirements Document (TRD)

Project: IPSEC-VPN — AI-Powered IPsec VPN Protocol Analyzer

Document Version: 1.0 **Status:** Draft **Date:** August 31, 2026 **Primary Stack:** Python (AI Layer) · Go/Golang (Backend) · PostgreSQL (DB) · Redis (Cache/Queue) · Docker (Containerization)

1. Architecture Overview

IPSEC-VPN is composed of five logical layers, deployed as independent Dockerized services orchestrated via Docker Compose (Kubernetes-ready for later phases):



2. Technology Stack

Layer	Technology	Notes
AI/ML Layer	Python 3.12, scikit-learn / XGBoost, PyTorch (for sequence models if needed), Scapy, pyshark/tshark bindings	Model training, feature extraction, inference microservice exposed via gRPC/REST (FastAPI)
Backend Services	Go 1.23+, Gin or Fiber (HTTP), gRPC, go-redis, pgx/GORM	API gateway, orchestration, job dispatch, auth
Database	PostgreSQL 16	Primary relational store; JSONB columns for flexible SA/feature

		metadata
Cache/Queue	Redis 7	Job queue (or Redis Streams), caching, session/token store
Containerization	Docker, Docker Compose (Kubernetes manifests as stretch goal)	All services, DB, cache, and testbed nodes run as containers
Testbed IPsec Engines	strongSwan, Libreswan	Configurable via templated <code>ipsec.conf</code> / <code>swanctl.conf</code>
Packet Capture	tcpdump, libpcap, Scapy	Raw capture and programmatic packet crafting/parsing
Frontend Dashboard	React + TypeScript, Recharts/D3 for visualizations	Served as static assets via Go backend or separate Nginx container
Report Generation	Python (WeasyPrint/ReportLab for PDF, Jinja2 for HTML templating)	Invoked by worker pool
Auth	JWT (access/refresh), bcrypt/argon2 password hashing	RBAC roles: Admin, Analyst, Viewer
CI/CD	GitHub Actions (or GitLab CI)	Lint, test, build, container image publish
Observability	Prometheus + Grafana, structured logging (zap for Go, structlog for Python)	Metrics on job latency, classification accuracy drift, queue depth

3. Service Breakdown

3.1 API Gateway / Backend (Go)

- Exposes REST endpoints for: auth, capture upload, job status, report retrieval, testbed management, user/role management.
- Validates and authenticates requests (JWT middleware), enforces RBAC.
- Publishes analysis jobs to Redis queue; consumes job-completion events to update Postgres and notify dashboard (WebSocket/SSE).
- Talks to the AI Layer via internal gRPC for synchronous inference calls where needed (e.g., quick single-session classification), and via queued jobs for batch/large PCAP analysis.

3.2 AI Layer (Python, FastAPI + gRPC)

- **Feature Extraction Module:** Parses PCAP (via pyshark/tshark or Scapy) into structured features:
 - IKE negotiation fields (exchange type, proposals, transforms, DH group, nonce sizes, identity payloads).
 - ESP/AH header fields (SPI, sequence numbers, ICV length, padding, next header).
 - Flow-level statistics (packet size distribution, inter-arrival time, flow duration, byte volume, directionality) for traffic-type inference.
- **Protocol Identification Models:**
 - Rule-based classifier for cases where IKE negotiation is visible (deterministic mapping from SA proposal payloads to algorithm/mode/DH group — high confidence).
 - ML classifier (gradient boosted trees) for cases requiring inference from side-channel/packet-level features (lower confidence, always reported as such).
- **Traffic-Type Inference Model:** Supervised classifier (e.g., XGBoost or small 1D-CNN/LSTM on flow sequences) trained on labeled flows from the testbed, using only metadata (size/timing) — never payload content.
- **Security Scoring Engine:** Deterministic rules engine (weighted scoring table, versioned/configurable YAML) evaluating crypto strength, PFS, key lifetime, replay protection, and compliance baseline mapping; produces the 0–100 risk score and severity banding.
- **Confidence Scoring:** Every model output includes a calibrated confidence score (e.g., softmax

probability or rule-certainty tag); aggregated into an overall AI Confidence Score for the report.

- Exposes `/classify`, `/score`, `/infer-traffic-type` endpoints (internal only, called by Go backend/workers).

3.3 Worker Pool

- Language: Go for orchestration-heavy tasks (testbed provisioning, capture ingestion), Python for AI-dependent tasks (feature extraction, report rendering) — both consume from the same Redis-backed job queue using a shared job schema.
- Job types: `capture_ingest`, `protocol_classify`, `security_score`, `report_generate`, `testbed_provision`, `testbed_capture`.
- Idempotent, retryable job design with dead-letter handling for failed jobs.

3.4 VPN Testbed Controller

- Provisions paired IPsec endpoint containers (strongSwan/Libreswan) with parameterized configs (mode, cipher, DH group, PFS, IKE version, IPv4/IPv6).
- Uses Docker networks / Linux network namespaces to simulate two (or more) sites; optional netem for latency/jitter simulation.
- Triggers traffic generator containers (e.g., SIPp for VoIP/SIP-RTP simulation, curl/locust for web traffic, swaks for SMTP, ping for ICMP, ffmpeg/RTP streaming for video) post-tunnel-establishment.
- Captures traffic via a tcpdump sidecar attached to the tunnel interface; auto-tags the resulting PCAP with the exact configuration used (ground truth) and stores it with metadata in Postgres/object storage.

4. Data Model (PostgreSQL — core tables)

```
-- Users & auth
users(id, username, email, password_hash, role, created_at)

-- Captures
captures(id, uploaded_by, filename, storage_path, source_type ENUM('upload', 'live', 'testbed'),
capture_started_at, capture_ended_at, status, created_at)

-- Testbed configurations & runs
testbed_configs(id, name, mode ENUM('tunnel', 'transport'), ike_version, cipher, auth_algo,
dh_group, pfs_enabled BOOLEAN, ip_version ENUM('ipv4', 'ipv6'),
traffic_profile, created_by, created_at)
testbed_runs(id, testbed_config_id, capture_id, status, started_at, completed_at)

-- Analysis jobs
jobs(id, capture_id, job_type, status, priority, retries, error_message,
created_at, started_at, completed_at)

-- Protocol classification results
classification_results(id, capture_id, ike_version, mode, encryption_algo, auth_algo,
dh_group, pfs_detected BOOLEAN, sa_lifetime_seconds, sa_lifetime_bytes,
replay_protection BOOLEAN, confidence_score FLOAT, raw_features JSONB,
model_version, created_at)

-- Traffic type inference
traffic_type_results(id, capture_id, flow_id, predicted_type, confidence_score,
feature_summary JSONB, model_version, created_at)

-- Security assessment / scoring
security_assessments(id, capture_id, risk_score INT, severity
ENUM('low', 'medium', 'high', 'critical'),
crypto_strength_score INT, compliance_baseline, compliance_pass BOOLEAN,
findings JSONB, threat_matrix JSONB, ai_confidence_score FLOAT,
created_at)

-- Reports
```

```
reports(id, capture_id, assessment_id, report_type ENUM('executive','technical'),
        format ENUM('pdf','html'), storage_path, generated_at)

-- Audit log
audit_log(id, user_id, action, resource_type, resource_id, metadata JSONB, created_at)
```

Indexes on `captures.status`, `jobs.status`, `classification_results.capture_id`, and `security_assessments.capture_id` for query performance. JSONB columns used for flexible/evolving feature and finding structures.

5. Redis Usage

- **Job Queue:** Redis Streams (consumer groups) for job dispatch/consumption across Go and Python workers.
- **Cache:** Cached dashboard aggregate stats (recent risk scores, job counts) with short TTL.
- **Session/Token Store:** Refresh token blacklist / active session tracking for JWT auth.

6. API Design (representative REST endpoints, Go backend)

```
POST /api/v1/auth/login
POST /api/v1/auth/refresh

POST /api/v1/captures/upload          # multipart PCAP upload
GET  /api/v1/captures/{id}
GET  /api/v1/captures/{id}/status

POST /api/v1/testbed/configs          # create testbed configuration
POST /api/v1/testbed/runs             # trigger provisioning + capture
GET  /api/v1/testbed/runs/{id}

GET  /api/v1/captures/{id}/classification
GET  /api/v1/captures/{id}/security-assessment
GET  /api/v1/captures/{id}/reports?type=executive|technical&format=pdf|html

GET  /api/v1/dashboard/summary
GET  /api/v1/audit-log
```

Internal (AI layer, not public-facing):

```
POST /internal/ai/classify
POST /internal/ai/infer-traffic-type
POST /internal/ai/score
```

7. AI/ML Pipeline Details

1. **Data Generation:** Testbed Controller systematically sweeps configuration permutations (mode × cipher × DH group × PFS × IP version × traffic profile) to produce a balanced, labeled dataset.
2. **Feature Extraction:** Deterministic parser (tshark/Scapy-based) converts PCAPs into structured feature vectors + flow-level time series.
3. **Model Training:** Offline training pipeline (Python, scikit-learn/XGBoost, optionally PyTorch for sequence-based traffic-type inference) with train/val/test split stratified by configuration class; tracked via MLflow (or simple versioned artifact storage) for reproducibility.
4. **Model Serving:** Trained model artifacts loaded by the FastAPI inference service; model version recorded alongside every prediction in Postgres for traceability.
5. **Evaluation:** Held-out test set accuracy/precision/recall/F1 per class, confusion matrices, calibration curves for confidence scores — published as part of technical documentation deliverable.
6. **Retraining (future phase):** Scheduled or triggered retraining job as new labeled data is

generated from additional testbed runs.

8. Security Assessment Rules Engine

- Configuration-driven (YAML/JSON) scoring table mapping algorithm/parameter → weight/penalty, versioned in the repo and referenced by `model_version / ruleset_version` in `security_assessments`.
- Baselines supported: NIST SP 800-77 Rev.1 guidance, IETF RFC 8247 (algorithm recommendations) — selectable per assessment.
- Deterministic and independent of the ML layer (auditable/explainable), combined with ML classification confidence to produce the final `ai_confidence_score`.

9. Deployment Architecture

- `docker-compose.yml` defining: `backend` (Go), `ai-service` (Python/FastAPI), `worker` (Go+Python), `postgres`, `redis`, `frontend` (Nginx-served React build), and a separate `testbed` compose profile for on-demand IPsec lab containers (isolated Docker network, privileged/NET_ADMIN capability scoped narrowly to testbed containers only).
- Environment-based configuration via `.env` files; secrets not committed to source control.
- Volumes: `pcap-storage`, `report-storage`, `model-artifacts`, `pg-data`.
- Kubernetes manifests (Helm chart) as a Phase 4 stretch deliverable for production-style deployment.

10. Security Considerations (of the platform itself)

- Testbed network isolated from production/host networks by default (dedicated Docker bridge network, no host network mode).
- Uploaded PCAPs scanned for basic sanity (file type/size limits) before parsing to reduce parser-abuse risk.
- RBAC enforced at the API layer; audit log captures all report generation and testbed provisioning actions.
- Secrets (DB credentials, JWT signing keys) managed via environment variables / secret files, never hardcoded; rotated per environment.
- TLS termination for all external-facing endpoints (dashboard, API).

11. Testing Strategy

Type	Scope
Unit tests	Go services (handlers, job logic), Python feature extraction & scoring rules
Integration tests	End-to-end job flow: upload → classify → score → report, against a test Postgres/Redis instance
Model evaluation tests	Accuracy/F1 thresholds enforced in CI against a fixed held-out labeled dataset
Testbed validation tests	Verify each configuration permutation establishes successfully and produces expected ground-truth-labeled traffic
Load tests	Concurrent job submission against worker pool to validate scalability targets

12. Milestone Mapping to PRD Phases

TRD Component	PRD Phase
Testbed Controller, PCAP ingestion, rule-based classifier, technical report (HTML)	Phase 1 (MVP)

ML traffic-type model, risk scoring engine, executive report, PDF export, dashboard v1	Phase 2
Live capture ingestion, RBAC, public API, audit log, selectable compliance baselines	Phase 3
Kubernetes/Helm deployment, retraining pipeline, additional baselines	Phase 4 (stretch)